

# Sistemas embarcados

Prof. Décio Rennó



O que é um  
sistema  
embarcado?



# Sistema embarcado?

- Sistema dedicado
- Microprocessado
- Hardware simplificado
- Baixo custo
- Recursos escassos



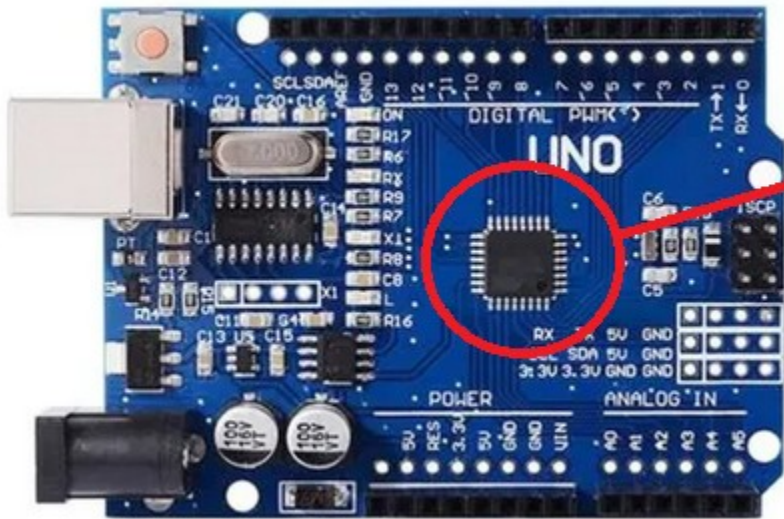
# O que é um sistema embarcado?

- Sistema dedicado



# O que é um sistema embarcado?

- Microprocessado



Microprocessador



# O que é um sistema embarcado?

- Hardware simplificado
- Baixo custo
- Recursos escassos

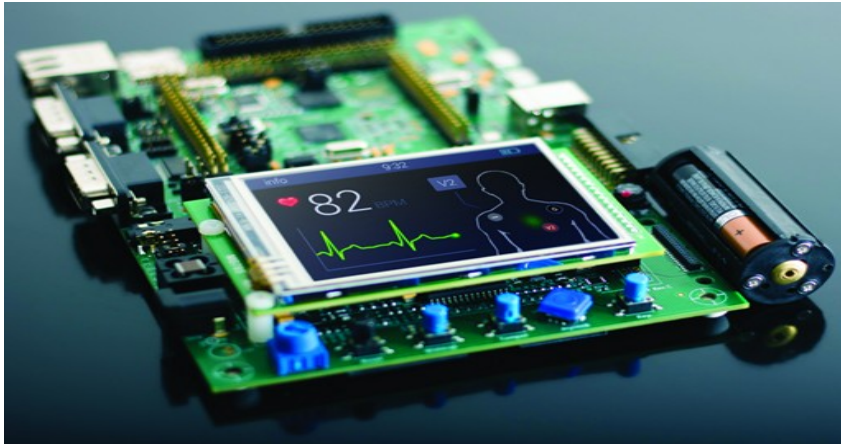


# Sistemas embarcados

- Desafios:
  - Mão de obra qualificada
    - Conhecimento em várias áreas
  - Projeto em times
    - Hardware x Software
  - Normas
  - Concorrência



# Projeto de sistemas embarcados



Circuito eletrônico

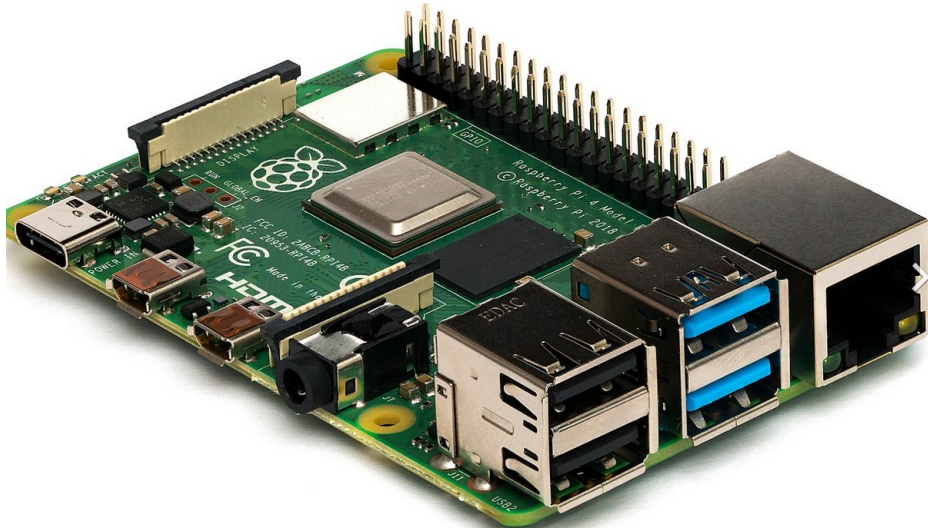
```
18 int ilength;
19 double dblTemp;
20 bool again = true;
21
22 while (again) {
23     in = -1;
24     again = false;
25     getline(cin, sInput);
26     system("cls");
27     stringstream(sInput) >> dblTemp;
28     stringlength = sInput.length();
29     if (ilength < 4) {
30         again = true;
31         continue;
32     } else if (sInput[ilength - 3] != '.') {
33         again = true;
34         continue;
35     } while (++in < ilength) {
36         if (isdigit(sInput[in])) {
37             continue;
38         } if (in == (ilength - 3)) {
```

Software



Documentação

# Sistemas embarcados

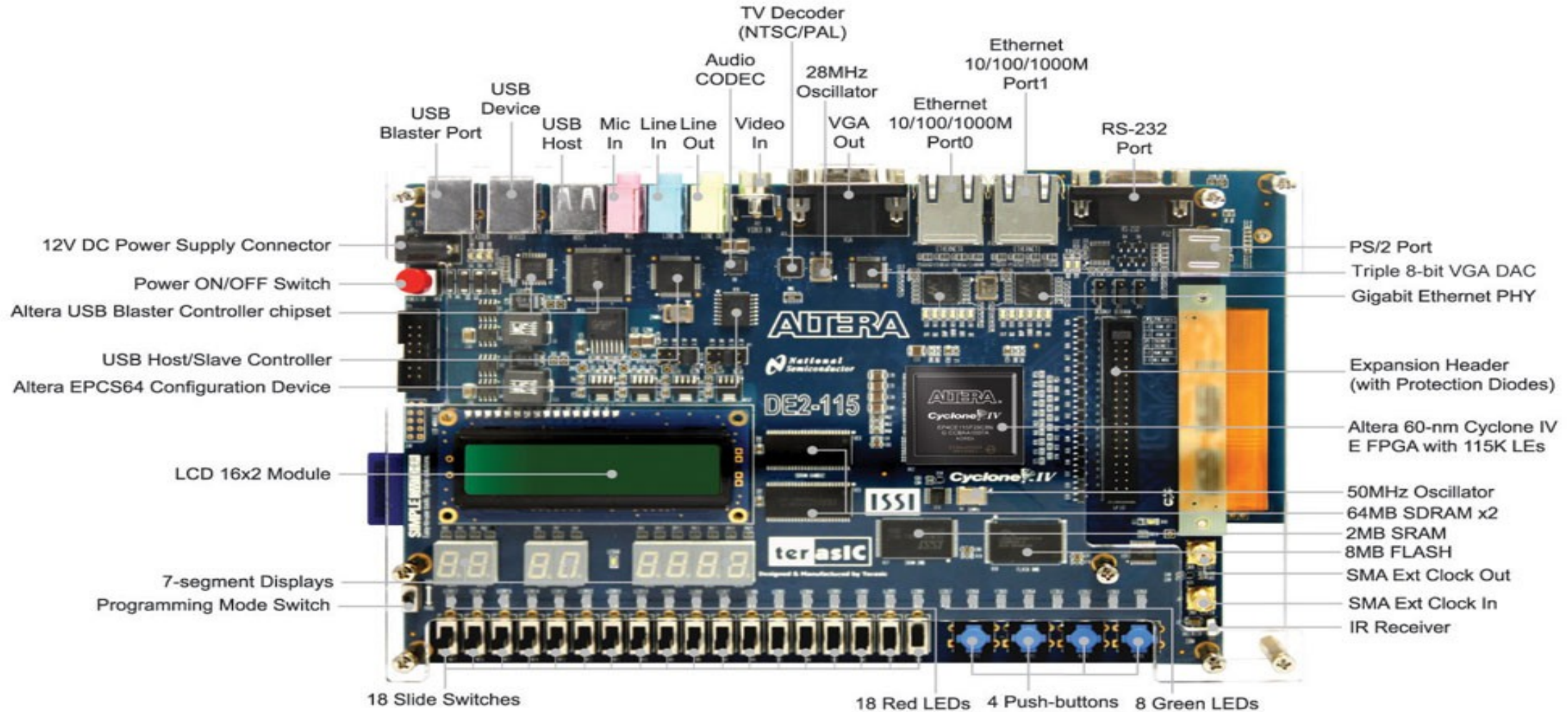


Raspberry Pi

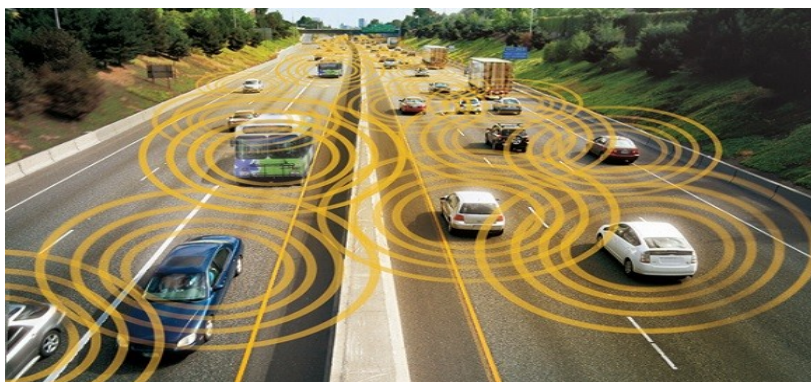
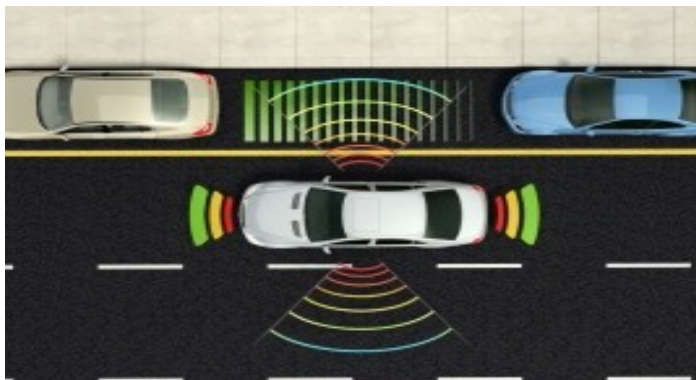


Smart Phone

# FPGA



# Aplicações

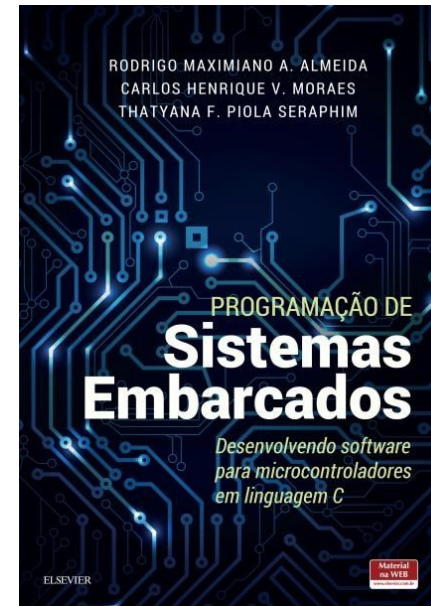


# Planejamento do curso



# Referências bibliográficas

- Programação de Sistemas Embarcados. Almeida, R.M.A., Moraes, C.H.V, Seraphim T.F.P., Elsevier 2016
- C completo e total. Herbert Schildt. Makron Books. 3a Edição. 1997.
- Datasheets e manuais



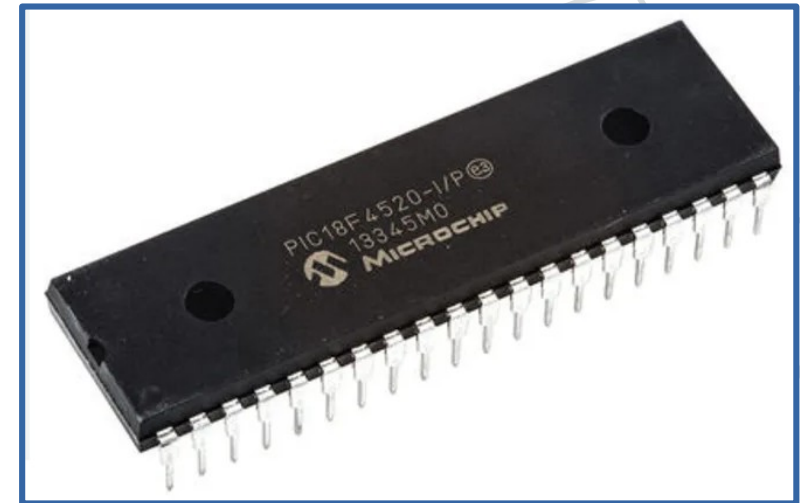
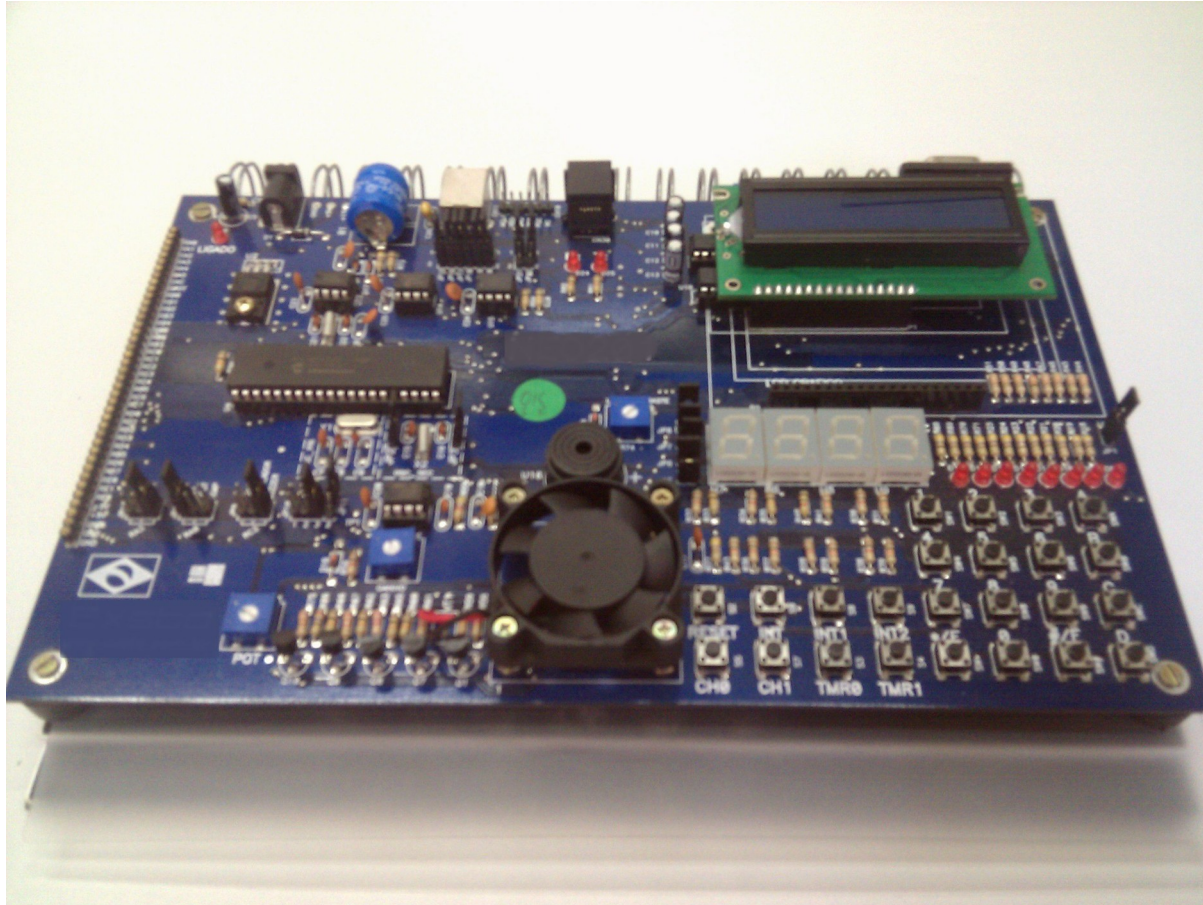
# Laboratório



Hardware e software utilizados

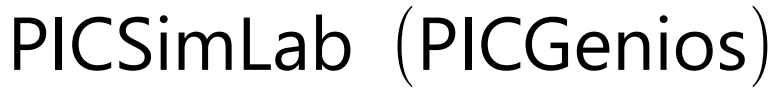


# Kit de desenvolvimento



Módulo Exsto com processador PIC18f4520

## A stylized logo consisting of a large gear-like shape on the left, a square in the center, and a diagonal line crossing through the square from the bottom-left to the top-right. The entire logo is rendered in a light gray color.



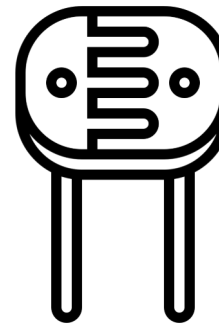
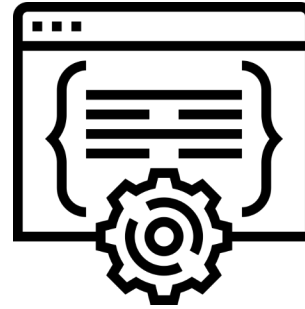
# Ferramentas a serem utilizadas:

- M P L A B X (versão 6.10+)
- C o m p i l a d o r X C 8 (versão 2.41+)
- P I C S i m L a b (versão 0.9.1)



# Requisitos

- Linguagem C
  - Estruturas de decisão: if/switch
  - Estruturas de repetição: for
  - Vetores
  - Funções
- Circuitos
  - Tensão x Corrente x Resistência
  - Fontes de Tensão
  - Divisor de tensão

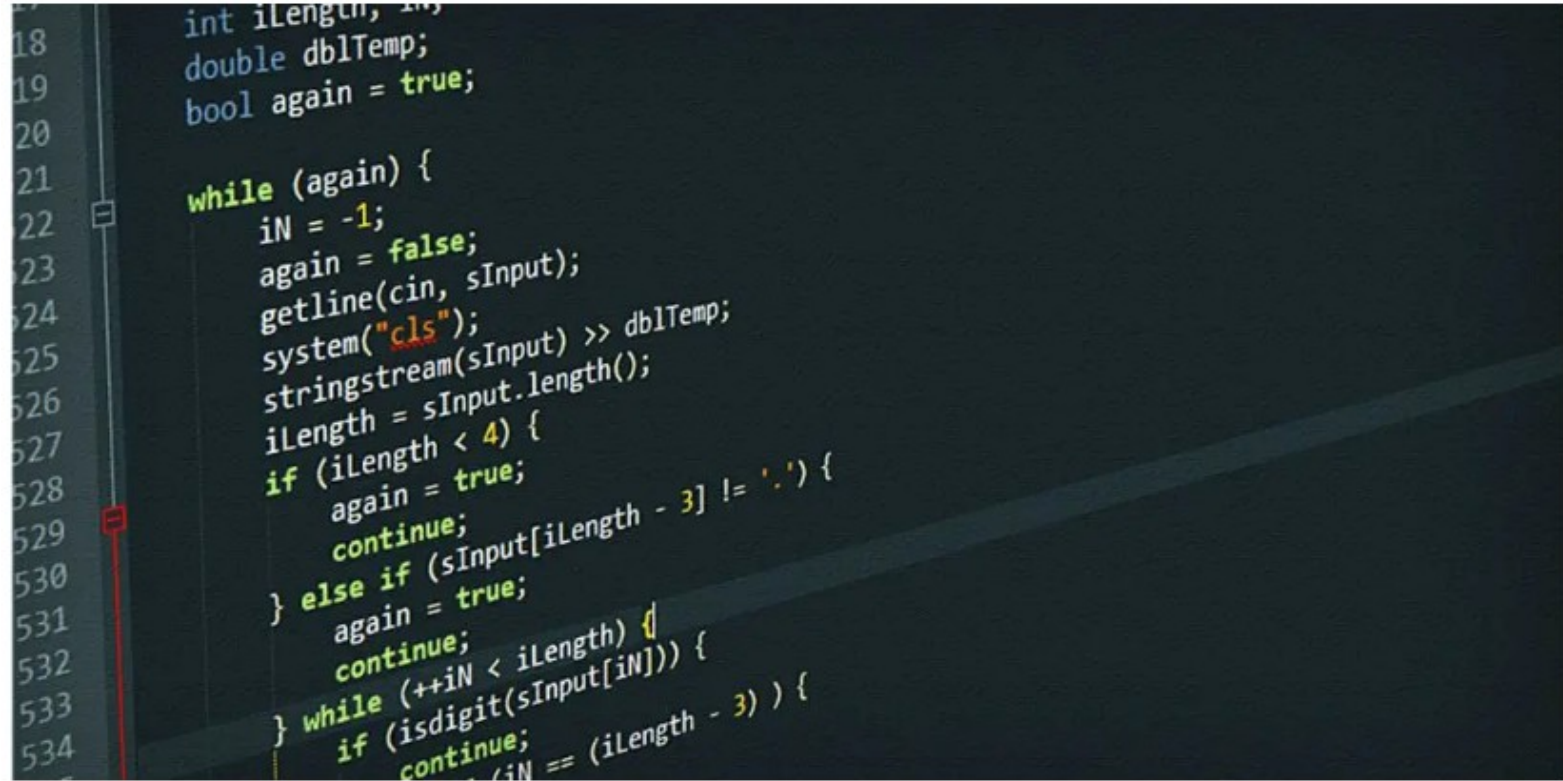


# Linguagem C



# Linguagens utilizadas

- Linguagem C
- Linguagem C++
- Assembly
- Python
- HTML
- Java



```
18 int iLength, iN;  
19 double dblTemp;  
20 bool again = true;  
21  
22 while (again) {  
23     iN = -1;  
24     again = false;  
25     getline(cin, sInput);  
26     system("cls");  
27     stringstream(sInput) >> dblTemp;  
28     iLength = sInput.length();  
29     if (iLength < 4) {  
30         again = true;  
31         continue;  
32     } else if (sInput[iLength - 3] != '.') {  
33         again = true;  
34         continue;  
35     } while (++iN < iLength) {  
36         if (isdigit(sInput[iN])) {  
37             continue;  
38         } if (iN == (iLength - 3)) {
```

# Comparativo

## Linguagem C

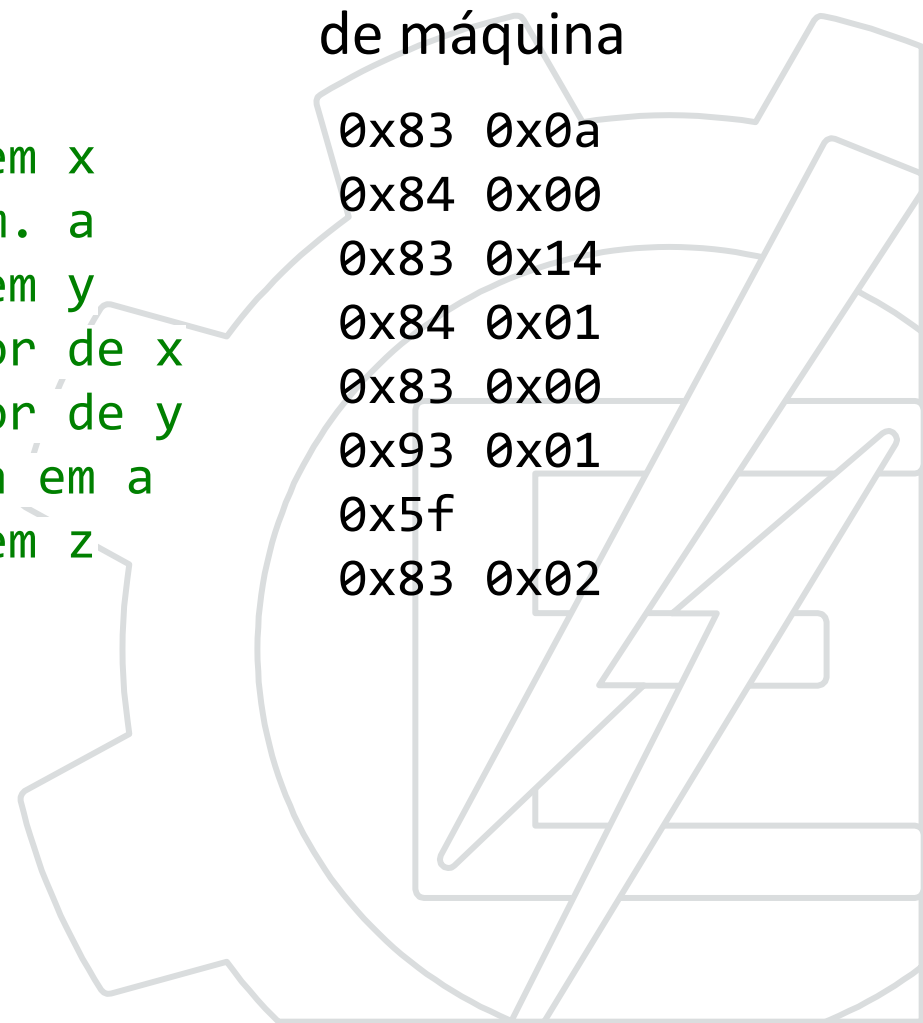
```
x = 10;  
y = 20;  
z = x*y;
```

## Linguagem ASM

```
ldaa 10 // carrega 10 no acum. a  
staa x  // salva o valor do acum. a em x  
ldaa 20 // carrega o valor 20 no acum. a  
staa y  // salva o valor do acum. a em y  
ldaa x  // carrega o acum. a com valor de x  
ldab y  // carrega o acum. b com valor de y  
mulab  // mult. acum. a por b, salva em a  
staa z  // salva o valor do acum. a em z
```

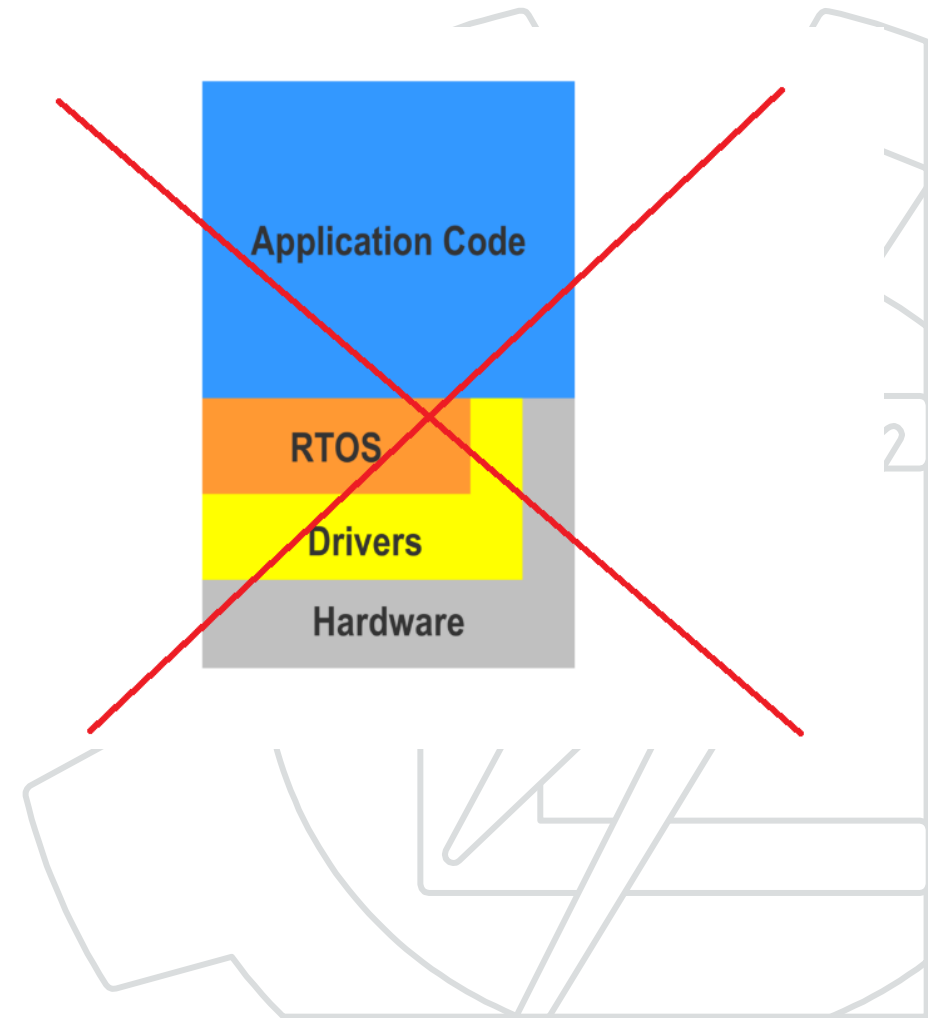
## Linguagem de máquina

```
0x83 0x0a  
0x84 0x00  
0x83 0x14  
0x84 0x01  
0x83 0x00  
0x93 0x01  
0x5f  
0x83 0x02
```



# Programação em Bare-metal

- Programando em baixo nível (nível de hardware)
- Sem uso de abstrações
- Sem sistema operacional
- O código está muito próximo da eletrônica



# Compilação em C

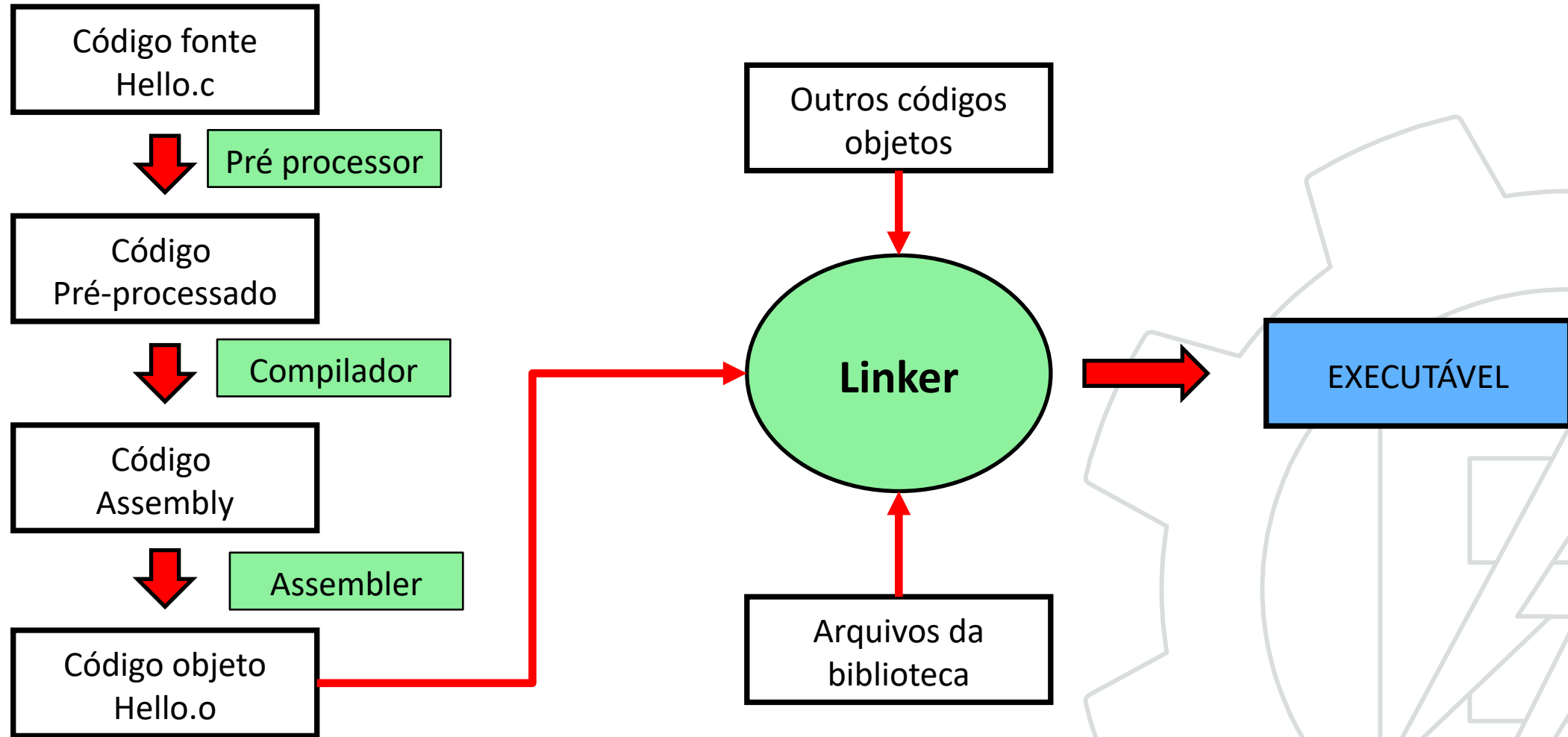


# Etapas da compilação

- Pré processamento
- Compilador
- Assemblador
- Linker



# Etapas da compilação



# O pré processador

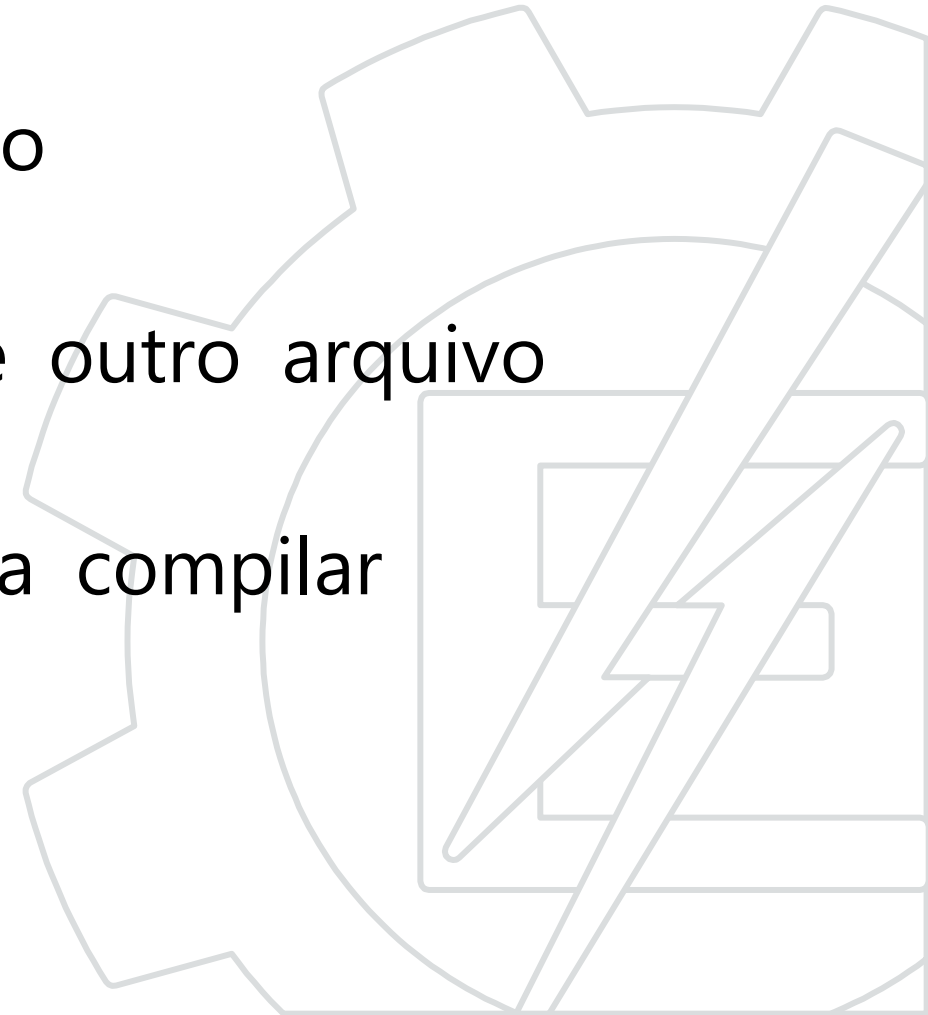
Uma ferramenta de substituição de textos

`#define` – Substitui uma macro

`#include` – Inclui um texto de outro arquivo

`#if` – Testa uma condição para compilar

Outros...



# O pré processador

## Exemplos de diretivas do pré processador

```
#define MAX_ARRAY_LENGTH 20
```

```
#include <stdio.h>  
#include "myheader.h"
```

```
#ifdef DEBUG  
    /* Your debugging statements here */  
#endif
```

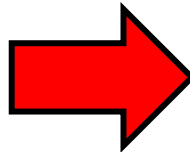


# O Compilador e Assemblador

Transforma texto em código de máquina

```
#include <stdio.h>

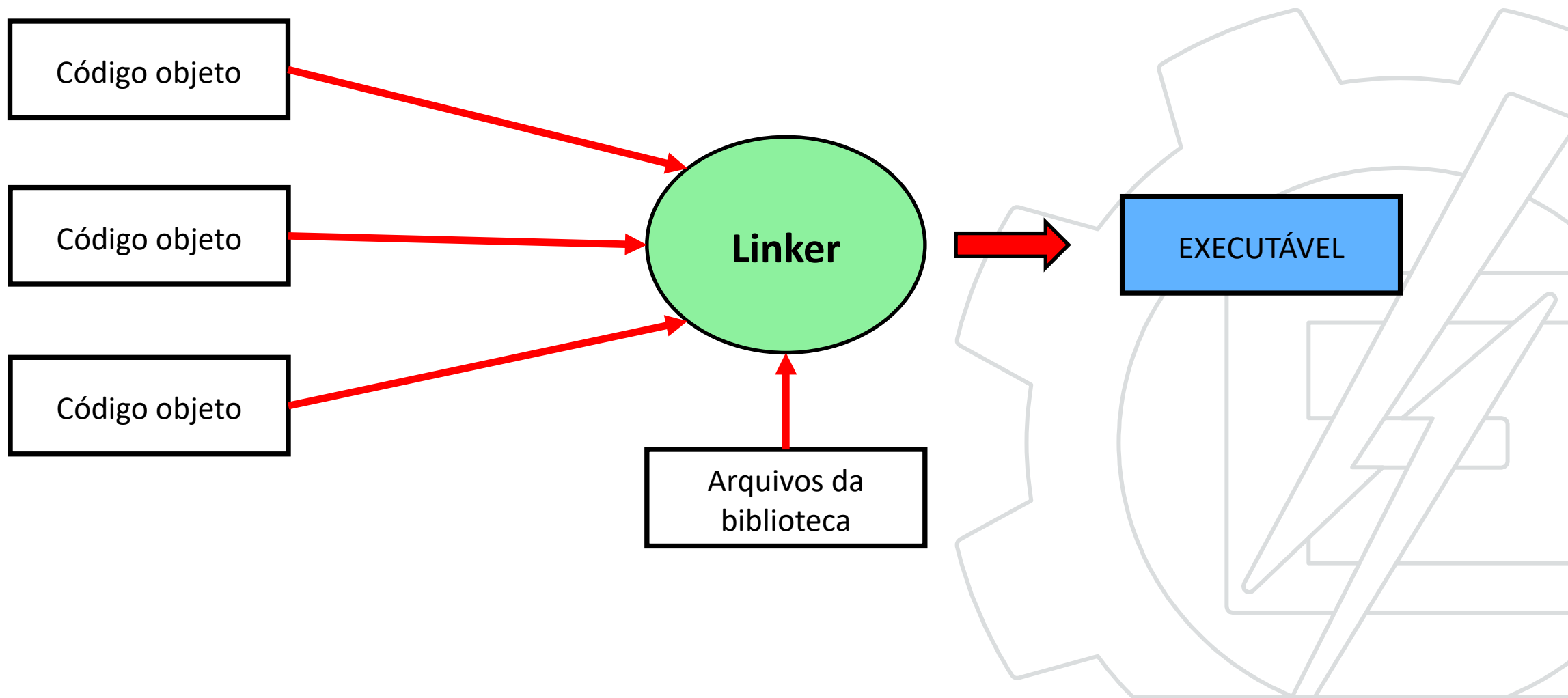
void main(void) {
    printf("Hello World");
    return;
}
```



| Arquivo  | Editar      | Abas        | Ajuda       |                              |
|----------|-------------|-------------|-------------|------------------------------|
| 00000000 | B4 03 CD 10 | B0 01 B3 0A | B9 0E 00 BD | 13 7C B4 13 ..... ..         |
| 00000010 | CD 10 F4 48 | 65 6C 6C 6F | 20 57 6F 72 | 6C 64 21 0D ...Hello World!. |
| 00000020 | 0A 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000030 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000040 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000050 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000060 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000070 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000080 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 00000090 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 000000A0 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 000000B0 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 000000C0 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |
| 000000D0 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 | 00 00 00 00 .....            |

O arquivo gerado é chamado de código objeto

# O Linker



# Programando em C



# Linguagem C

- Coleção de funções

Criadas pelo programador

Utilizadas através de biblioteca

- Os programas em linguagem C possuem

Diretivas de pré processamento

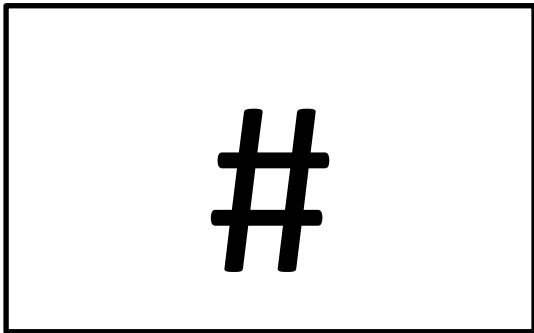
Definição de variáveis

Definição de funções

Etc.



# As diretivas de pré-processamento



- Mensagens do programador ao compilador com tarefas para antes ou no momento da compilação
- as diretivas são iniciadas pelo caractere # (hash)
- as diretivas mais comuns são **#include** e **#define**

# Diretivas de pré-compilação

- A diretiva `#define` permite que o código fonte seja modificado antes de ser compilado.

```
#define CONST 15
void main(void){
    printf("%d", CONST * 3);
}

//no processo de compilação
void main(void){
    printf("%d", 15 * 3);
    //é possível: printf("%d", 45);
}
```



# Compilação condicional

Verificando um #define  
antes de compilar

#ifdef *nome*

#else

#endif

Exemplo de uso:

```
void ImprimirTemp(char valor){  
    #ifdef LCD  
        Imprime_LCD(valor);  
    #else  
        if (valor > 30) led = 1;  
        else led = 0;  
    #endif //LCD  
}
```

# Compilação condicional

## Outro exemplo

```
void MostraSaidaPadrao(){  
#ifdef SERIAL  
    char *msg = "SERIAL";  
#else  
    char *msg = "LCD";  
#endif  
    printf("%s",msg);  
}
```

Com selecionar entre Serial e LCD:

```
#include <stdio.h>  
#define SERIAL  
void main(void){  
    MostraSaidaPadrao();  
}
```

```
#include <stdio.h>  
//#define SERIAL  
void main(void){  
    MostraSaidaPadrao();  
}
```

# Includes

- Especifica ao compilador novas funções, tipos e macros que estão disponíveis em outros arquivos

```
#include <nome do arquivo>  
#include "nome do arquivo"
```



# A diferença entre os Includes

Header files: são os arquivos com extensão .h que contém as definições de tipos, dados e declarações de várias funções já prontas

`#include <nome do arquivo>`

- O arquivo é procurado no diretório definido pelo compilador C

`#include "nome do arquivo":`

- O arquivo é procurado no diretório atual do projeto.

# Os Includes não podem repetir

Na grande maioria dos casos é necessário fazer uma verificação para garantir que o include não seja chamado duas ou mais vezes

Cuidado



Imagem: [www.graphicsfactory.com](http://www.graphicsfactory.com)

```
#ifndef STDIO_H
#define STDIO_H

//Protótipos, definições, ...

#endif // Fim do STDIO_H
```

# Funções

Blocos de comandos a serem executados

- A função pode, ou não, receber valores.
- As funções podem, se necessário, retornar um valor.
- A função “main” é onde começa o programa.
- Não há ordem obrigatória para codificar as funções

# Definição de funções

## Exemplo:

A definição da função possui:

- O tipo de dado que ela retorna
- O nome da função
- O que ela recebe
- O que ela faz

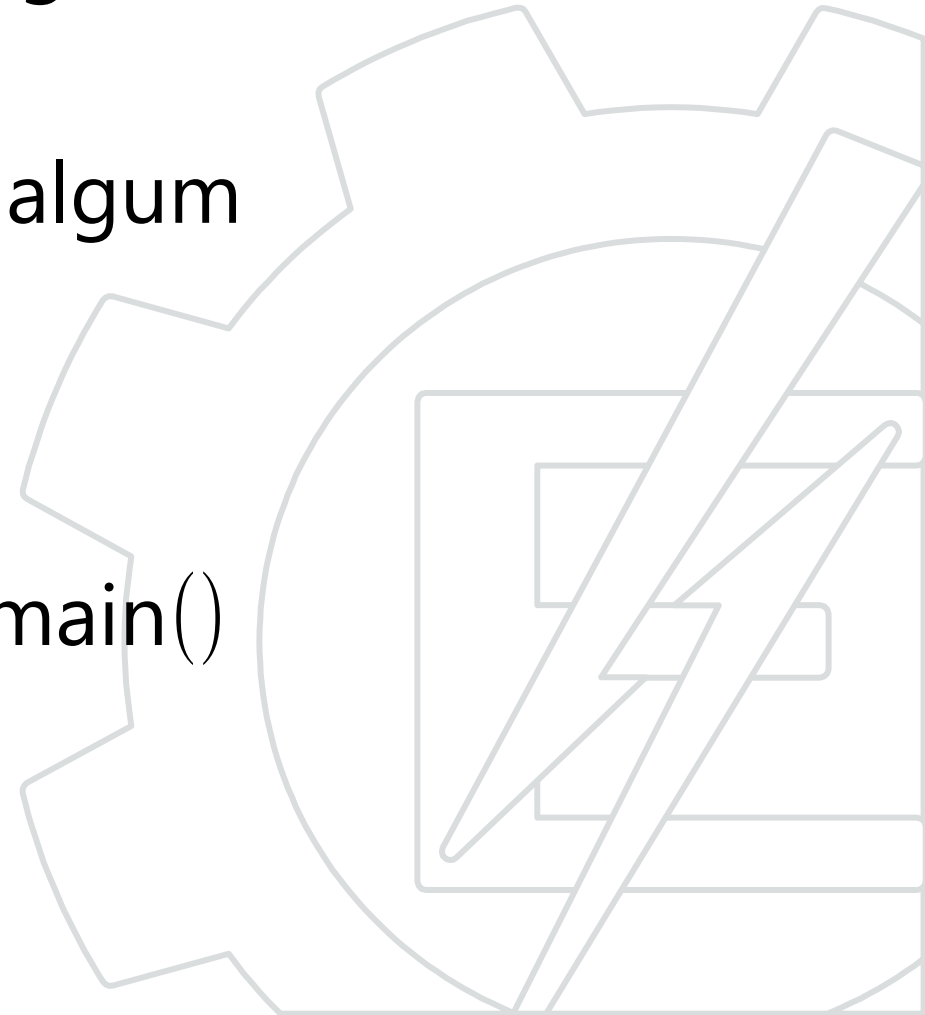
```
int main(int argc, char *argv[]){  
    return 0;  
} //end main  
  
void funcao01(char var){  
    ...  
} //end funcao01  
  
void funcao02(void){  
    ...  
} //end funcao02
```

# Função main



# Função main

- Indica o ponto de início de um programa
- O linker aloca a função `main()` em algum lugar disponível na memória
- O linker atualiza o vetor de reset colocando um pulo para a função `main()`



# Função main

- Em sistemas embarcados a função main é a primeira a ser executada, portanto não pode receber parâmetro e não retorna nada.
- Deve ser declarada como:

```
void main(void){  
    //aqui entra o código do programa  
}
```



# Função main

- Geralmente os sistemas embarcados são feitos para serem executados continuamente
- As rotinas/funções devem ser pensadas para serem executadas ciclicamente
- O sistema só para quando desligado
- Nesta situação devemos utilizar loops infinitos



# Loop infinito?



# Função main

## Exemplos de loop infinito em software embarcado

```
void main (void){  
    for(;;){  
        //aqui entra o  
        //código principal  
        //e nunca termina  
    }  
}
```

```
void main (void){  
    while(1){  
        //aqui entra o  
        //código principal  
        //e nunca termina  
    }  
}
```

Até a próxima

